דף סיכום בחינה

מזהה סטודנט:

מזהה קורס: 20594   שם קורס: מערכות הפעלה

מספר שאלה	ניקוד מירבי	ציון
1	25.00	24.00
2	17.00	11.00
3	13.00	9.00
4	5.00	2.00
5	5.00	4.00
6	5.00	5.00
7	5.00	5.00
8	5.00	1.00
9	5.00	5.00
10	5.00	5.00
11	5.00	5.00
12	5.00	0.00

ציון בחינה סופי : 76.00

הבחינה הבדוקה בעמודים הבאים

מספר התלמיד הנבחן
רשום את כל תשע הספרות

האוניברסיטה הפתוחה

ח' בתמוז תשפ"ב

מס' שאלון - 525

7
ביולי 2022

סמסטר 2022ב

מס' מועד 89

20594 / 4

שאלון בחינת גמר

20594 - מערכות הפעלה

משך בחינה: 3 שעות

בשאלון זה **12** עמודים

מבנה הבחינה:

קראו בעיון לפני שתתחילו בפתרון הבחינה!

א. המבחן מורכב משלושה חלקים.

ב. בחלקים א ו - ב מופיעות שאלות פתוחות. ענו תשובות מלאות, בכתב קריא ובקיצור נמרץ. אין חובה להשתמש בכל השורות המוקצות לצורך התשובות, אך אין לחרוג מהמקום המוקצה.

ג. בחלק ג (שאלות אמריקאיות) עליכם לבחור בכל פעם בתשובה יחידה מבין התשובות המוצעות ולהקיף בעיגול את אות התשובה שבחרתם.

את התשובות לכל השאלות יש לכתוב בשאלון הבחינה.

חומר עזר:
כל חומר עזר אסור בשימוש, פרט למחשבון, שאינו אוצר מידע.

בהצלחה !!!

החזירו
למשגיח את השאלון
וכל עזר אחר שקיבלתם בתוך מחברת התשובות

89.52.12 M2 שאלון 525

~~גחלה~~

~~lock (p) {~~
~~if (v == 0)~~
~~v = m(p)~~

לכן כניסה

*

```
lock (lock) {
  if (lock == 0)
      lock = 1
      return 0;
  else return lock
}
```

lock = v

~~lock (v, p) {~~
~~v = m(p)~~

```
unlock {
    lock --;
}
```

~~un lock (v,~~
~~if~~

```
lock
while (
```

[redacted]
8
5
5
5
5

הנחיות למבחן

בחינה בעקרונות מערכות הפעלה

קראו בעיון לפני שתתחילו בפתרון הבחינה!

1. בחלקים א׳ ו ב׳ מופיעות שאלות פתוחות. ענו **תשובות מלאות, בכתב קריא ובקיצור נמרץ**. אין חובה להשתמש בכל השורות המוקצות לצורך התשובות, אך **אין לחרוג מהמקום המוקצה.**
2. בחלק ג׳ (שאלות אמריקאיות) עליכם לבחור בכל פעם בתשובה **יחידה הנכונה ביותר** מבין התשובות.
3. כל חומר עזר אסור לשימוש בזמן הבחינה פרט למחשבון פשוט
4. משך הבחינה - שלוש שעות.

בהצלחה!

חלק א (55 נקודות)

שאלה 1 (25 נקודות)

מחשב חדש שנמצא בשלבי פיתוח מבוסס מעבד תומך בשתי פקודות מכונה מיוחדות לתיאום בין תהליכונים(THREADS). הקומפיילר תומך בפקודות המכונה הללו באמצעות מקרואים מיוחדים שמתורגמים לפקודות המכונה.

פקודת מכונה אחת מפעילים בעזרת המקרו m שהוא שקול לאופרטור * על מצביעים, כלומר v=m(p) שקול לטעינת הערך ששמור בכתובת p לתוך המשתנה v.

המקרו השני נקרא st והוא מיוחד יותר. הביטוי $r = st(v,p)$ מנסה לכתוב את הערך v לכתובת p בזיכרון.

st ⇒ *p = v

הכתיבה מצליחה ומוחזר הערך 1, אם ורק אם התהליכון שמבצע את $st(v,p)$ ביצע לפני כן m(p) על הכתובת p וזו הפעולה האחרונה שבוצעה על הכתובת p ואף תהליכון אחר לא כתב ערך לכתובת p מאז שהתהליכון הזה ביצע m(p) .

בכל מקרה אחר, כלומר אם התהליכון לא ביצע את m או אם תהליכון אחר כתב לכתובת p מאז שהוא ביצע את m (בין אם בעזרת st ובין אם בעזרת השמה בכל דרך), הכתיבה נכשלת (הערך v לא נכתב לכתובת p) והמקרו מחזיר את הערך 0 . במילים אחרות, הכתיבה מצליחה רק אם הערך שכתוב בזיכרון בעת ניסיון הכתיבה הוא אותו הערך שהתהליכון בעצמו קרא בעזרת ה m ואף תהליכון אחר לא כתב לתוכו מאז.

(המשך השאלה בעמוד הבא)

lock (p) {   int turn = *p

while (

int v

lock (p) {
    v = m(p)
    while (st(v,p) == 0) ;   √
    retur

while (== 0);

~~הגדרה~~ ואז
כדי לנווט

lock
    if (st(v,p)) == 0

✗. מימוש

*p = 0
lock (p)
    if (st(v,p))
        return;
    else
        while (st(v,p)) ;

lock = *p

unlock

lock

א. **(15 נק')** ממשו שגרות lock(p) ו unlock(p) למניעה הדדית, כאשר p הוא מצביע לשלם שמייצג את המנעול. המימוש צריך להשתמש במקרואים(MACROS) המתוארים בשאלה.

לא תשובה
נמחק
בחינה

✓

ב. **(10 נק')** מוגדר מבנה נתונים של רשימה מקושרת של מבנים מהטיפוס:

struct item { int data; struct item *next }

עם מצביע struct item *head לראש הרשימה. ממשו שגרה insert(int d) שמכניסה איבר חדש לרשימה שערך השדה data בו הוא d, וזאת תוך שימוש במקרואים m ו st. אסור להשתמש במנעולים ואסור להגדיר משתנים נוספים (פרט למשתנים אוטומטיים בשגרה).

השגרה צריכה לפעול נכון גם בסביבה שבה יש עוד תהליכונים שמנסים לפעול על הרשימה במקביל.

```
Insert (int d) {
  lock (p)
  int data = m(&d)
  while ( st ( data, &d) ==0 );
  item i = new Item();
  item* helper = *head
  while ( helper != null ) {
      helper = head -> next
  }
  i = helper
  i -> data = data
  unlock (p)
}
```

אם נכשל, ימשיך להיכשל

24
(1)

$$\frac{10 \cdot 2^{20} B}{Sec} / 1000 =$$

4

$$\frac{2^{20}}{100}$$

1MB אחד. $\frac{1}{10}$ Sec

1KB $\frac{1}{100}$ Sec

10 B 4 1 $\frac{1}{1000}$ Sc $\frac{4}{10} \cdot 1000$

B 1 1 MS

$\frac{4}{100}$ Sec

40.

---

2 וו. לוקח 10 שניות להעביר 10MB לכן לוקח שניה
להעביר 1MB ← לוקח $\frac{1}{10}$ שניות ל1KB אז
$\frac{4}{10}$ שניות לגדל (4K) שהם 400 MS שניות.

$x \cdot 400 + 10 <$

$10 \cdot 400 + 10 < x \cdot 400 + 10x$

$4010 < 410x$ /

שאלה 2 (17 נקודות)

מערכת מחשב מסוימת משתמשת בדפים בגודל 4KB ודיסק **קשיח** לדפדוף שקצב העברת הנתונים שלו הוא 10MByte/sec וזמן חיפוש(SEEK) ממוצע בו הוא 10ms(10 אלפיות שנייה). כל המרחב הווירטואלי של כל תהליך שמור בדיסק בצורה רציפה. הניחו שבכל שגיאת דף(PAGE FAULT) קיימות 2 אפשרויות בלבד: המערכת תביא דף בודד או 10 דפים רצופים(כמו שנאמר הם שמורים בדיסק ברצף).

הערה: בחישוב לא צריך לקחת בחשבון את הזמנים של פעולות טיפול אחרות בשגיאת דף (PAGE FAULT) שלגביהם אין נתונים בשאלה.

$\frac{10 \cdot 40 + 10}{1 \cdot 40 + 10}$

א. **(4 נק')**

האם מנגנון הבאת דפים מראש(PREPAGING) עשוי להאט תכניות לעומת המערכת דפדוף לפי דרישה(ON DEMAND PAGING)? נמקו. אם עניתם שכן, מהי ההאטה המרבית(במה ואיך היא תלויה)? תכתבו תיאור מילולי בלי חישובים.

תשובה והסבר:

(הסבר בדף הלבן)

כיון שזמן העברה של דף הינו 400 ms וזמן חיפוש הינו 10ms לכן זמן מציין שזמן העברה גדול מזמן החיפוש פי 40 ואין צורך לבצע prepaging כי הזמן שאם אין pagefault נמוך מזמן העברה של 10 דפים מרווח שוליים אין צורך בהם. prepaging

$\frac{10 \cdot 400 + 10\ ms}{1 \cdot 400 + 10\ ms} = 9.7$

פי 9.7 יותר איטי. ON DEMAND

ב. **(7 נק')**

תארו מנגנון פשוט לקבלת החלטה ע"י מערכת הפעלה בזמן טיפול בשגיאת דף האם להביא דף בודד או 10 דפים. תארו את מבני הנתונים הדרושים למנגנות ואיזה גורם מעדכן את הנתונים האלה. המנגנון צריך לצרוך לא יותר ממספר קטן וקבוע של בתים בזיכרון לכל תהליך ולא יותר ממספר קטן וקבוע של פעולות חישוב בכל שגיאת דף. המנגנון חייב לקבל בד"כ החלטה נכונה במצבים שבם מתבצעות גישות אקראיות לזיכרון ראשי(למשל גישות לטבלת HASH) ובמצבים שבהם מתבצעות גישה רציפה לזיכרון (למשל חיפוש לינארי בטבלה גדולה). **מותר למנגנון לקבל החלטות שגויות במקרים אחרים.**

אם לא צריך,זמן קריאה ארוך

צריך להבחין בין גישה אקראית לרציפה

תשובה והסבר:

כדי שיהיה משתלם להשתמש בprepaging צריך ש x מספר הדפים

$400x + 10 > 10 \cdot 400 + 10 \Leftarrow x > 9.78$ ש $x > 10$ מ 10 דפים

זאת אומרת שהתהליך יהיה גדול מעל דפים רצופים לכן אם התהליך הוא גדול מ 40KB הדפים שמורים רצוף שווה לעשות prepaging

אחרת אין סיבה.

(המשך השאלה בעמוד הבא)

```
for (i=0, i<N, i++)
    S+ = A[i]·B[i]
```

ג. **(6 נק')**

האם המנגנון שתיארתם בסעיף ~~א'~~ ב' פועל בצורה נכונה כאשר התכנית מחשבת מכפלה פנימית של שני וקטורים ארוכים? (כלומר עבור הלולאה ;for (i=0; i<N; i++) s+=A[i] * B[i]).
הוכיחו שכן או הסבירו למה לא ותארו כיצד ניתן לשפר את המנגנון כך שיפעל נכון גם במקרה הזה.

תשובה והסבר :

שוב תלוי מה גודל N אם N שלא מגודל מס דפים
אז כן צריך לעשות prepaging עם לא עם פסוף להביא דף דף
~~הבא~~ ~~הי~~ ואז אם A נמצוי ~~ארחוק~~ ~~של~~ ~~פמות~~ מ ל' ד'
B ובאחד הם שדולים מס ~~פחות~~ דפים אז צריך לעשות
prepaging ולכן ~~נצטרך~~ נבאך הנפדים הדים A ו B מסודרים
ברצף לפני.

11
(2)

המשך הבחינה בעמוד הבא

נא לא לכתוב בשוליים

89.52.12 M2 שאלון 525

A מקבל גובל B ו C

D רוצה להכנס ויין מקום
נכנס לרווח בין A ו B
ויוצ A רוצה לגדול

שאלה 3 (13 נקודות)

כידוע, אחת הבעיות הפוגעות בביצועי מערכת הקבצים היא קבצים שאינם שמורים באופן רציף על הדיסק הקשיח.

א. **(3 נק')**

נציע שיטה חדשה להקצאת מקום לקובץ - "שיטת הרווחים". השיטה תשמש לצמצום ממדי הבעיה ע"י השארת מקום ריק (רווח) בגודל קבוע לאחר כל קובץ, כך שאם הקובץ צריך לגדול, הוא יכול לגדול באופן רציף. המקום השמור לאחר כל קובץ מיועד לקובץ עצמו, אבל מותר להשתמש במקום זה עבור הקצאה לקבצים אחרים אם אין ברירה אחרת (כלומר רק במקרה שאין יותר מקום פנוי על הדיסק חוץ ממקומות השמורים לגדילה של קבצים).

תנו דוגמה פשוטה למקרה בו, כשמשתמשים ב"שיטת הרווחים" אחד הקבצים אינו שמור באופן רציף.

תשובה והסבר:

נניח כי יש שלושה קבצים A, B ו C שכל אחד מקבל מקום ועוד חצי עבור הרווח, ואז מגיע ~~קובץ~~ קובץ D ואין מקום פנוי בסופו לכן נכנס לרווח בין A ל B ואז ~~A מבקש~~ ~~לכן הוא מקבל מקום בין~~ C & נמחק ואז A מבקש לגדול לכן מקבל את המקום של C.

ב. **(5 נק')**

הועלתה הצעה הבאה ("שיטת החצי"): הקובץ הראשון ישמר בתחילת הדיסק. בהמשך, בכל פעם שנוצר קובץ חדש, נאתר את השטח הפנוי הרצוף הגדול ביותר על הדיסק, ונשמור את הקובץ החדש החל מאמצע שטח זה.

הסבירו מה היתרון לשמירת קובץ חדש באמצע הקטע הפנוי הרצוף הגדול ביותר ולא בתחילתו?

האם שיטה זו תמנע לחלוטין מצב בו קובץ שמור בכמה חלקים נפרדים (כלומר, האם כל הקבצים יהיו שמורים ברצף)? נמקו.

תשובה והסבר:

היתרון בלשמור קובץ חדש באמצע קטע פנוי הרצוף הגדול ביותר ולא בתחילתו היא מכיוון שזה נותן אפשרות לשני הקבצים לגדול ברווח בלי צורך לחלק את הקובץ לכמה חלקים אבל זה לא מבטיח. נניח כי קובץ A הוא ראשון בזיכרון ואז B נשמר מהחצי הפנוי ואז C מגיע ואין לו מקום לכן נשמר ברווח בין A ל B ואז A מבקש לגדול ויש מקום אחרי B (שמספיק לו ואז לא הספיק לו C) לכן A לא יהיה ברצף.

(המשך השאלה בעמוד הבא)

[illegible]

ג. **(5 נק')**

נניח שאנו משתמשים ב"שיטת החצי" כדי למקם קבצים חדשים. מבין השיטות למימוש קבצים שלמדנו, איזו המתאימה ביותר לצרכנו? השיטות שלמדנו הן מימוש קבצים באמצעות הקצאה רציפה (contiguous allocation), מימוש קבצים באמצעות הקצאה בשיטת רשימה מקושרת (linked list allocation), מימוש קבצים ברשימה מקושרת באמצעות אינדקס (מבוססת i-nodes)?. נמקו.

תשובה והסבר:

אם אין מספיק מקום?

הכי טוב יהיה הקצאה רציפה, מכיוון שגודל קובץ הוא

~~הקבוע~~ ידוע מההתחלה ? וגם הגודל של החצי קבוע. וגם אין

צורך ברשימה מקושרת לניהול שטחים ריקים, כי המטרה

בשיטת החצי שהקובץ ישמר בצורה רציפה, גם Inode — גם אפשר ברצף

לא רלוונטי כי אנחנו מעדיפים ~~ש~~ שהקובץ ישמר רציף.

9

(3)

המשך הבחינה בעמוד הבא

נא לא לכתוב בשוליים

getppid()	pid	
1	2.	סבא
2	3	בן
3	0	נכד

חלק ב (25 נקודות)

לפניכם חמש שאלות. יש לענות על כולן.

5 נקודות לשאלה; 25 נקודות לחלק זה.

שאלה 4 (5 נקודות)

הסבירו מה תהיה המשמעות אם במערכת XV6 שתומכת בקונטיינרים לא בוצע בכלל כתיבת "cpu+" לקובץ cgroup.subtree_control : קונטיינר לא ייווצר, ייווצר, אך לא יתפקד בכלל כי לא יקבל את זמן CPU או ייווצר, אך לא יתפקד כמצופה ?

2
(4)

תשובה והסבר :

הקונטיינר יווצר אך לעולם לא ירוץ כי מערכת ניהול התהליך לא יכיר בו ולכן ~~לא יקבל לעולם זמן~~ ~~ה~~ cpu .

שאלה 5 (5 נקודות)

במערכת XV6 קיימת קריאת מערכת (KILL (PID שמאפשרת להרוג את התהליך ע״פ PID שלו.

פונקציה ()getppid מחזיקה את ה PID של האב(FATHER), ל PROCESS עם PID=1 , היא מחזירה 0.

מה יהיה הפלט אחרי הרצת הפקודות הבאות במערכת XV6 שתומכת בקונטיינרים?

```
unshare(PID_NS);
int pid = fork();
if(pid == 0) {                                         בן
  pid = fork();
  if(pid == 0) {                                       (נכד
   sleep(999); // long sleep
   kill(getppid());                                    בן מת
   printf(stdout, "GrandSon PID=%d \n", getpid());     Gra   PID=3
   }
  else {                                               בן
    sleep(9999); // very  long sleep
printf(stdout, "Son PID=%d \n", getpid());
   }
 }
else {                                                 אבא
  sleep(99999); //very very sleep enough for creating and running all son processes
  printf(stdout, "Father still alive \n");
 }
```


נא לא לכתוב בשוליים

89.52.12 M2 שאלון 525

[illegible]

ניתן להניח שכל קריאות מערכת מצליחות ושלא נוצרו תהליכים נוספים במערכת מרגע הרצת הקוד(חוץ מאלה שהקוד יוצר) ושמערכת מקצה PID עוקב כל פעם שנוצר תהליך חדש.

4 (5)

2

תשובה והסבר:

הפלט יהיה: הנכד יודפס: Granson PID = 3

הסבא יודפס: Father still alive \n

מכיוון שהנכד הרג את הסבא של בשורה 7 ומכיוון שהבן

ישן יותר זמן מהנכד. ולכן הבן מת ולא

הספיק להרוג את האבא שלו.

שאלה 6 (5 נקודות)

מה יודפס בעקבות הרצת קטע קוד הבא בתוך מערכת xv6 שתומכת בקונטיינרים, ניתן להניח שכל קריאות מערכת מצליחות:

```
int pid = fork();
if (pid == 0) {
  mkdir ("dirS");
  mount ("internal_fs_a","dirS",0); //mount a preformatted internal_fs_a
  int fd = open ("dirS/file.txt",O_CREATE|O_RDWR);
  write (fd ,"123456789\n",10);
  close (fd);
 }
if (pid > 0) {
sleep (10000); // wait for child to finish
unshare (MOUNT_NS);
 }
sleep (10000); // both processes will sleep
if( pid == 0) {
   if (open ("dirS/file.txt",O_RDONLY) < 0) printf (stderr, "son failed to open\n");
   else printf (stderr, "son succeeded\n");
 }
If (pid>0) {
   if (open ("dirS/file.txt",O_RDONLY) < 0) printf (stderr, "father failed to open\n");
   else printf (stderr, "father succeeded\n");
}
```

הבן

האבא

5 (6)

תשובה והסבר:

יקום fd יכתב המחרוזת 123456789 לתוך הקובץ יסגר התהליך

ישנו ואז ~~...~~ לא ישן בנתיים הבן גם יוכל לישון ואז האבא

יחזור ואז הבן יתעורר יכתוב "Son succeded" הוא לא ישלח

unshare יוכל הבן כבר סיים ואז האבא גם יכתוב

"father succeeded"

המשך הבחינה בעמוד הבא

525 שאלון 89.52.12 M2

נא לא לכתוב בשוליים

[illegible]

שאלה 7 (5 נקודות)

לדיסק בעל 40 גלילים(CYLINDERS), כאשר הזרוע נמצאת בגליל 13 (בכיוון תנועה ושירות בקשות כלפי מעלה- הגליל בעל המספר הגבוה ביותר), מגיעה סדרת בקשות קלט/פלט לגלילים הבאים (משמאל לימין):

22, 11, 36, 15, 39, 1

איזה מאלגוריתמי תזמון הדיסק הבאים SSF - החיפוש הקצר קודם , SCAN - אלגוריתם המעלית

ו C-SCAN - סריקה מחזורית בכיוון למעלה בלבד יתבצע תוך השגת מינימום מרחק חיפוש כוללי? הראו את החישובים בבירור.

חישוב והסבר:

~~SSF → |13-15| + |11-15| + |15-22| + |22-36| + |36-39| + |39-1| = 68~~

SCAN → |13-15| + |15-22| + |22-36| + |36-39| + |39-11| + |11-1| = 64

C-SCAN → |13-15| + |15-22| + |22-36| + |36-39| + |39-1| + |1-11| = 74

SSF → |13-15| + |15-11| + |11-1| + |1-22| + |22-36| + |36-39| = 54

האלגוריתם הכי טוב יהיה SSF!

5
(7)

שאלה 8 (5 נקודות)

איזו תכונה חשובה נדרשת למעבד בשביל שיהיה ניתן לממש עליו את ביצועי הווירטואליזציה בשיטת Trap-and-Simulate?

~~המעבד צריך לאפשר תמיכה בהתקנים ושמעבדים ישירות עם הזכרון הראשי בלי צורך לעבור דרכו.~~

ההסבר לא נכון

1
(8)

המשך הבחינה בעמוד הבא

נא לא לכתוב בשוליים

[illegible]

חלק ג (20 נקודות)

לפניכם 4 שאלות. יש לענות על כולן.

(5 נקודות לשאלה; 20 נקודות לחלק זה).

שאלה 9 (5 נקודות)

אחד ההבדלים בין פונקציות ספריה בשפת C (כגון printf) לבין קריאות מערכת (system calls) ב- Linux הוא:

א. רק למנהל המערכת (super user) מותר להשתמש בקריאות מערכת, בפונקציות ספריה יכול להשתמש כל אחד

ב. **כל** קריאות המערכת הן חלק מפונקציות הספרייה של שפת C

ג. **כל** פונקציות הספרייה ממומשות בעזרת קריאות מערכת

ד. כל התשובות הקודמות אינן נכונות

5 (9)

שאלה 10 (5 נקודות)

מערכת הקבצים של מערכת הפעלה מסוימת משתמשת בשיטת ה I-node.

- גודל הבלוק במערכת הקבצים הוא 1 KBytes
- כתובת הבלוק היא 4 בתים (bytes)
- 10 שדות של ה I-node יכולים להחזיק ישירות כתובת הבלוק בדיסק
- שדה נוסף אחד נועד להחזיק כתובת של ה single indirect block
- עוד שדה נוסף אחד נועד להחזיק כתובת של ה double indirect block
- ועוד שדה נוסף אחד נועד להחזיק כתובת של ה triple indirect block

$\frac{2^{10}}{2^2} = 2^8 = 256$

חשבו מהו הגודל המקסימלי של קובץ במערכת הקבצים הזו?

הערה: לצורך החישוב 1 Kbyte = 1024 Byte , 1 GByte = 1024 * 1024 Kbyte

5 (10)

א. 15 GBytes

ב. 15.5 GBytes

ג. 16 GBytes

ד. 16.5 GBytes

0.5GB 16GB

$10 \cdot 2^{10} + 2^8 \cdot 2^{10} + 2^{16} \cdot 2^{10} + 2^{24} \cdot 2^{10}$

2^{34}

$2^4 \cdot 2^{30}$

$2^{20} \cdot 2^6$

המשך הבחינה בעמוד הבא

$$\frac{2^{20}}{2^{10}} \cdot 4 =$$

שאלה 11 (5 נקודות)

מהו החיסרון המובהק של שיטת ה- rendezvous לעומת ה- mailbox ב- message passing?

א. שיטת ה-rendezvous קשה יותר למימוש במערכת הפעלה.

ב. שיטת ה-rendezvous פחות גמישה מכיוון שהתהליכים (השולח והנמען) הופכים להיות מסונכרנים (תלויים זה בזה).

ג. בשיטת ה-rendezvous לא ניתן לממש מניעה הדדית בגישה לקטעים קריטיים (critical sections).

ד. שיטת ה-rendezvous גורמת לעומס קיצוני על מנגנון התזמון(SHEDULING)

שאלה 12 (5 נקודות)

נתון כי במערכת מסוימת המשתמשת בשיטת הדפדוף (paging) לצורך ניהול הזיכרון :

- גודל הדף הוא 4 K.
- הכתובת הוירטואלית הנה בת 20 סיביות (bits).
- גודל הזיכרון הפיזי הוא 1024 KB =1 MB.
- גודל שורה בטבלת הדפים (page table entry) הוא 4 בתים (bytes).

אם טבלת הדפים הנה טבלת הדפים המהופכת (inverted page table) מהו גודל הטבלה?

א. 2560 בתים.

ב. 512 בתים.

ג. 2048 בתים.

ד. 1024 בתים.

בהצלחה!

נא לא לכתוב בשוליים